

实验 1：建立仿真环境

欢迎大家来到机器人学基础课程。

咱们课程的课件和习题都是基于英文原版教材的，实验部分的内容原本也是英文，后来翻译成的中文。所以这部分内容读起来可能会有点生硬，请大家多担待。

希望大家在本课程都能有所收获，玩的开心。有任何问题，可以随时在群里提问。谢谢大家。

概述

在本实验中，我们将配置 Gazebo 仿真开发环境，调试并运行出一个 Turtlebot3 机器人和一个 ReactorX150 机械臂，方便后续实验过程中使用。

为了更好地照顾初学者，我们在实验安装部分提供了逐步指导。如果您已经对ROS和相关部分比较熟悉，可以跳过任何您觉得有把握的部分。

作业提交

- 需要提交的文件：
 - 提交几张截图，能说明您已成功安装所需软件，并启动了Turtlebot机器人和机械臂就可以。
- 评分标准：
 - 只要截图能证明你已经成功安装了软件并启动了机器人小车和机械臂，就可以了。

基础知识

在本课程中，我们会用到的机器人开发相关知识和技能有，Linux/Ubuntu系统，ROS，Python，Git/Github，以及对于部分同学可能会用到虚拟机。大家可以自行学习相关基础知识，同时在本课程中随着实验的进行，不断加深对这些知识技能的理解。

安装虚拟机

如果你有一台安装了 Linux 操作系统的笔记本电脑，或者可以双系统启动 Linux 操作系统，这是最佳的开发环境，就不需要安装虚拟机了。但是如果您不熟悉双系统或者不想承担潜在风险（计算机可能无法正常启动等），我们建议您使用 VMware 虚拟机。

- 如果您使用的是 Windows 笔记本电脑，请使用**VMware Workstation Pro**。如果您使用的是 Mac 笔记本电脑，请使用**VMware Fusion Pro**。这两款产品自 2024 年 5 月起免费供非商业和个人使用。下载页面在[这里](#)。您需要注册一个账户才能使用。

- 注意：我们不建议使用任何配备 M 系列芯片/处理器的苹果 Mac 笔记本电脑，因为在安装机械臂相关的软件程序库时可能会出现问题。
- 如果你更熟悉 VirtualBox（另一种虚拟机软件平台，完全免费开源），也可以使用它而不是 VMware。大家可以自行寻找相关使用教程和资源。

安装 Linux

安装好 VMware 后，让我们创建一个新的虚拟机并安装 Ubuntu 22.04。

- 从[官方网站](#)下载 Ubuntu 22.04 光盘镜像（64 位 PC 桌面）。
- 在 VMware 中创建一个新的虚拟机。
 - 典型配置
 - 选择下载的光盘镜像
 - 输入该虚拟机的一些信息
 - 再次输入名称
 - 请分配至少 50GB
 - 将虚拟磁盘存储为单个文件
 - 自定义硬件：请分配更多内存和 CPU 处理器以提高性能
 - 完成
- 现在你已经拥有了一台（虚拟）Linux 电脑。

注意：磁盘大小 50GB 不会立即分配，而是会随着你添加的内容逐渐增加。

安装 ROS2

ROS2 是本课程实验部分的核心软件。推荐两种安装方法：使用官方源直接安装，或者通过 Fishros 工具来安装。

方法 1：从官方源直接安装

按照 ROS2 Humble 官方安装指南（[英文版](#)，[中文版](#)），从 `apt` 安装 ROS2。

请务必安装桌面版（`ros-humble-desktop`）以获得完整功能。

 如果您在中国大陆，并且默认的 apt 软件源访问速度很慢，可以考虑把软件源换成清华大学提供的 [Ubuntu 镜像](#)。

方法 2：通过 Fishros 一键安装

Fishros 是 FishBot 社区开发的工具包，支持一键安装 ROS2（Humble、Foxy 等）。

- 适合中国大陆用户或喜欢简化安装过程的用户。
- 适合初学者，方便快速部署。

代码块

```
1  wget http://fishros.com/install -O fishros && . fishros
```

 如果你的电脑已经安装了其他版本的Ubuntu系统（比如Ubuntu 20 或者 24），并且无法安装Ubuntu 22的双系统，那么有两种选择。

- 1) 可以考虑使用 Ubuntu 22 的 Docker 环境来开发。详细的 Docker 安装步骤请参阅视频：[Bilibili - ROS2 Docker 安装教程](#)
- 2) 可以安装Ubuntu 24 或者 20 对应版本的 ROS2软件，而不是 ROS2 Humble，再在对应的环境下尝试安装 Turtlebot 和机械臂软件程序。除非您对自己的开发能力很有自信，并且熟悉Ubuntu和ROS2系统，否则我们不推荐这种方式，因为您在这个过程中需要自行解决很多软件兼容性的问题和潜在的系统 bug。

创建 ROS2 工作区

- 安装 `colcon`，它是 ROS2 中使用的构建工具，类似于 ROS1 中的 `catkin`。同时安装 `git`。

代码块

```
1  sudo apt update
2  sudo apt install python3-colcon-common-extensions git
```

- 运行以下命令来创建名为 `ros2_ws` 的 ROS 2 工作区并对其进行初始化。

代码块

```
1  mkdir -p ~/ros2_ws/src
2  cd ~/ros2_ws
3  colcon build
```

- 下面这些命令可以帮助我们配置 ROS2 系统的环境变量，只需要运行一次即可。他们所做的事情其实是把环境配置的命令添加到了 `.bashrc` 文件中，这样子每次打开一个新的系统终端的时候，这两个环境配置的命令就会被自动执行了。

代码块

```
1 echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
2 echo "source ~/ros2_ws/install/setup.bash" >> ~/.bashrc
3 source ~/.bashrc
```

- 接下来，让我们创建一个名为 `robotics101` 的新 ROS 软件包，供本课使用。

代码块

```
1 cd ~/ros2_ws/src
2 ros2 pkg create --build-type ament_python robotics101
```

- 现在，我们可以在这个工作区中构建这个 ROS 软件包。

代码块

```
1 cd ~/ros2_ws
2 colcon build --symlink-install
```

- 您已经完成了 ROS2 的基础教程。如需了解更多教程，可以访问以下网站：

[Tutorials — ROS 2 Documentation: Humble documentation](#)

[ROS 2 文档 — ROS 2 Documentation: Humble documentation](#)

[动手学ROS2](#)

安装 Gazebo 模拟器和 Turtlebot3 机器人

从现在起，我们假设您已经安装了 Ubuntu 22.04 和 ROS2 Humble。我们将按照[Turtlebot3 的官方说明](#)来设置模拟环境。（注意：以下步骤仅适用于 Ubuntu 22。如果您在使用其他的Ubuntu版本，请访问Turtlebot3的官方教程，寻找对应版本的安装步骤。）

- 安装 Gazebo

代码块

```
1 sudo apt install ros-humble-gazebo-*
```

- 安装 Cartographer（可选，非必须），用于 SLAM

代码块

```
1 sudo apt install ros-humble-cartographer
2 sudo apt install ros-humble-cartographer-ros
```

- 安装 Navigation2（可选，非必须），用于导航

代码块

```
1 sudo apt install ros-humble-navigation2
2 sudo apt install ros-humble-nav2-bringup
```

- 创建 Turtlebot3 工作区

代码块

```
1 mkdir -p ~/turtlebot3_ws/src
2 cd ~/turtlebot3_ws/src/
```

- 下载 Turtlebot3：按照您的网络情况，您可以选择从 GitHub 或 Gitee 下载 TurtleBot3 软件包。
 - 选项1：从 GitHub 下载（适合全球互联网稳定的用户，官方仓库的固定镜像版本）

代码块

```
1 git clone -b humble https://github.com/hanzheteng/turtlebot3
2 git clone -b humble https://github.com/hanzheteng/turtlebot3_msgs
3 git clone -b humble https://github.com/hanzheteng/turtlebot3_simulations
4 git clone -b humble https://github.com/hanzheteng/DynamixelSDK
```

- 选项2：从 Gitee 下载（我们创建的国内的镜像仓库）

代码块

```
1 git clone -b humble https://gitee.com/hanzheteng/turtlebot3
2 git clone -b humble https://gitee.com/hanzheteng/turtlebot3_msgs
3 git clone -b humble https://gitee.com/hanzheteng/turtlebot3_simulations
4 git clone -b humble https://gitee.com/hanzheteng/DynamixelSDK
```

- 安装 Turtlebot3

代码块

```
1 cd ~/turtlebot3_ws
2 colcon build --symlink-install
3 echo "source ~/turtlebot3_ws/install/setup.bash" >> ~/.bashrc
4 source ~/.bashrc
```

在 Gazebo 中运行 Turtlebot

- Turtlebot3 机器人有两种类型可选，我们选择其中的一种来做实验。运行下面的命令去指定我们要用到的机器人类型。（以下命令只需要运行一次，因为我们把这个环境配置命令添加到 `.bashrc` 文件后，每次打开一个新的系统终端的时候这个命令就会自动运行了。）

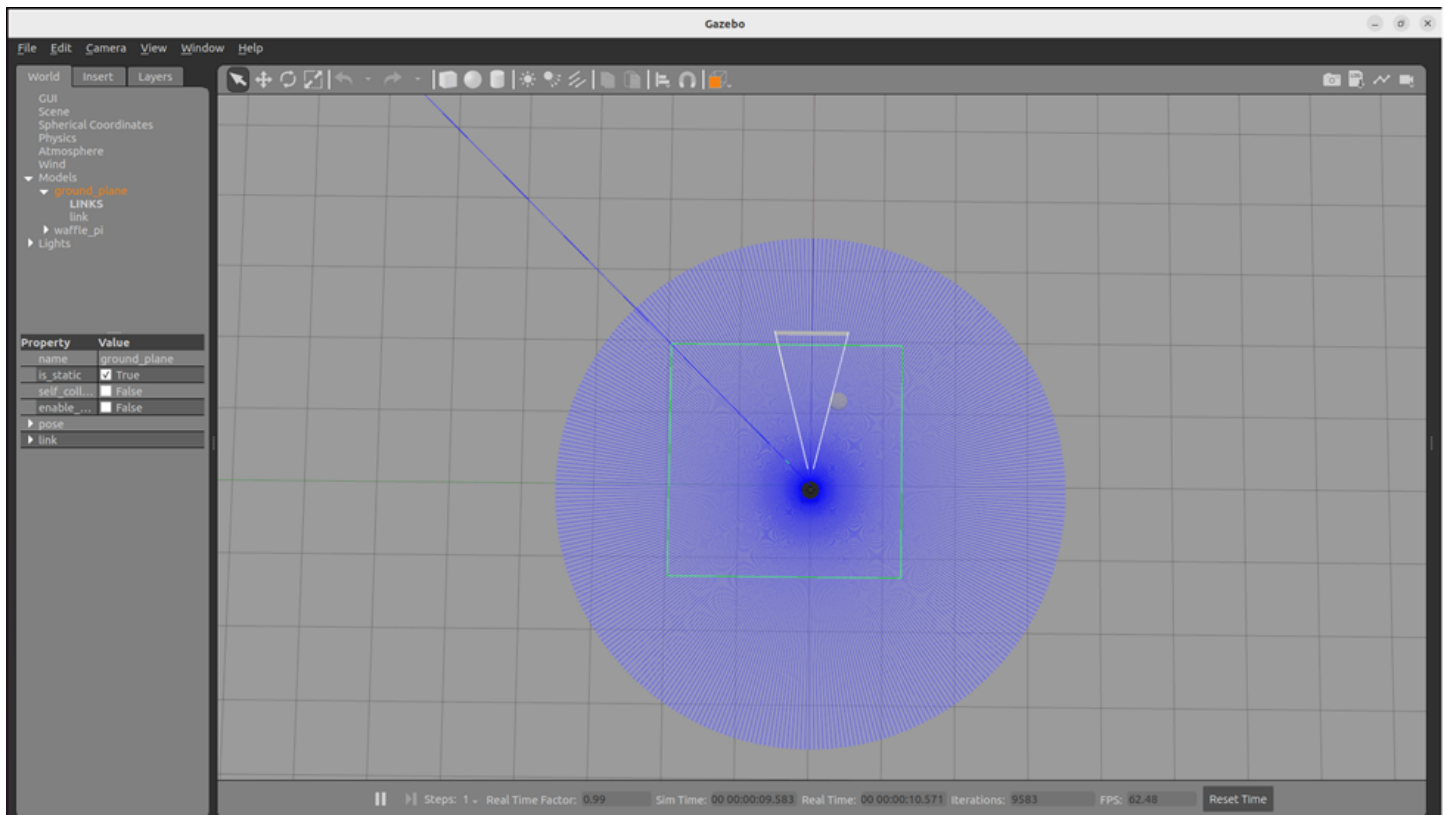
代码块

```
1 echo "export TURTLEBOT3_MODEL=waffle_pi" >> ~/.bashrc
2 echo "source /usr/share/gazebo/setup.sh" >> ~/.bashrc
3 source ~/.bashrc
```

- 现在可以启动 Gazebo 仿真器并生成一个 Turtlebot3 机器人了。第一次打开 Gazebo 时可能需要一些时间，因为它需要下载一些模型和世界环境。

代码块

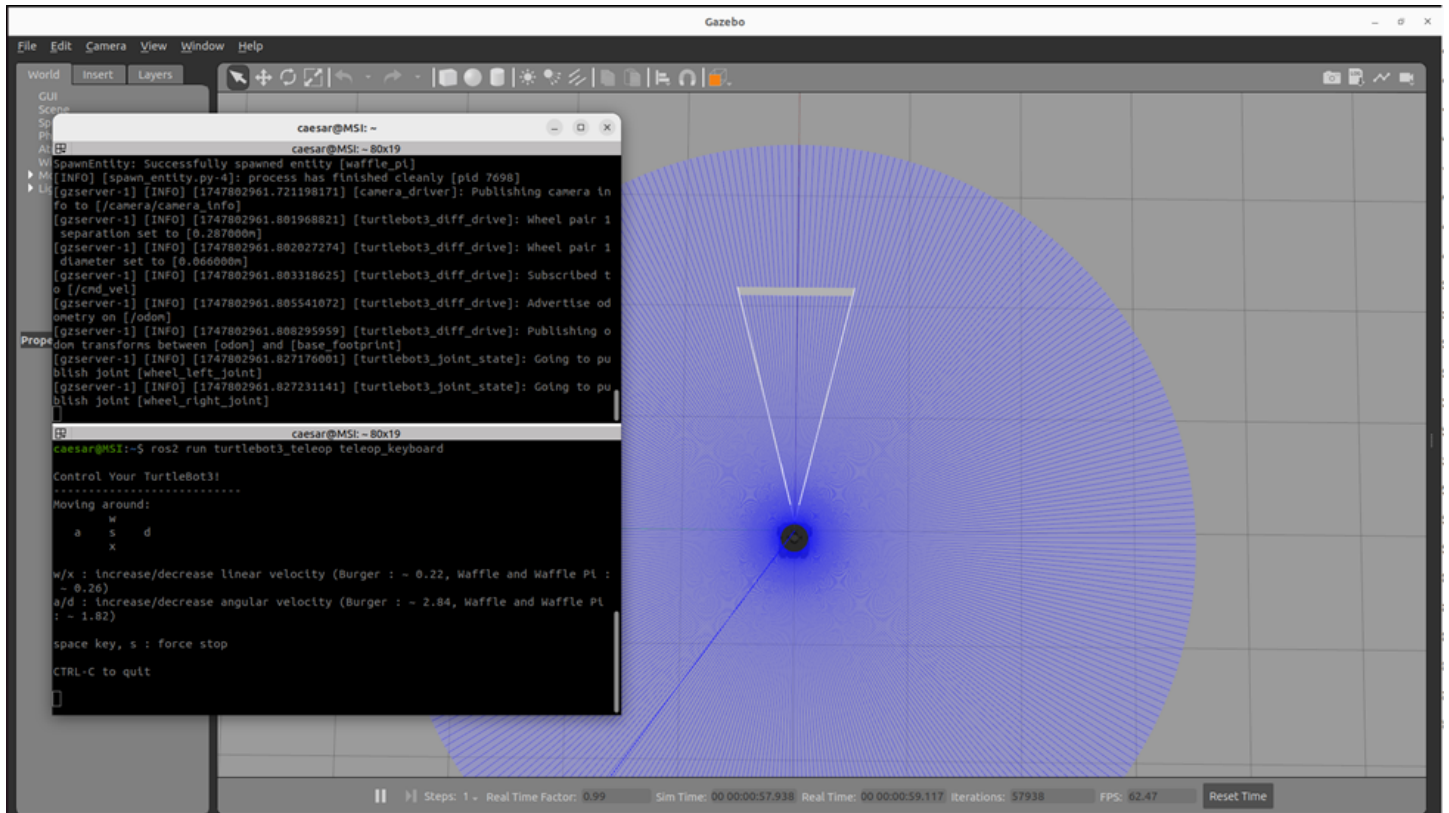
```
1 ros2 launch turtlebot3_gazebo empty_world.launch.py
```



- 在 Gazebo 中成功生成机器人后，我们可以打开一个新终端，运行以下命令启动键盘控制器。

代码块

```
1 ros2 run turtlebot3_teleop teleop_keyboard
```



- 保持远程操作终端打开（选中状态），现在就可以用键盘控制机器人了。该终端中的远程操作程序会接收您输入的任何按键，并将其转换为速度指令发送给机器人。
- 注意：如果要终止终端中运行的程序，请使用 `Ctrl + C` 并且等待一段时间（关闭 Gazebo 可能需要 10 秒钟）。如果终端关闭时没有正确终止程序（即程序仍在后台运行），下次运行时将出现 Gazebo 崩溃错误。

安装 RX-150 机械臂

方法 1：我们根据[官方教程](#)来安装 RX-150 机械臂系统。

代码块

```
1 sudo apt install curl
2 curl
  'https://raw.githubusercontent.com/Interbotix/interbotix_ros_manipulators/main/
  interbotix_ros_xsarms/install/amd64/xsarm_amd64_install.sh' >
  ~/xsarm_amd64_install.sh
3 chmod +x ~/xsarm_amd64_install.sh
4 ~/xsarm_amd64_install.sh -d humble
```

方法 2：从国内镜像源安装

代码块

```
1 sudo apt install wget
2 wget
  https://gitee.com/zhiyezhao/xsarm_installer_china/raw/master/xsarm_amd64_install_cn.sh
3 chmod +x ~/xsarm_amd64_install_cn.sh
4 ~/xsarm_amd64_install_cn.sh -d humble
```

- 对于安装脚本中提出的问题：
 - 无需安装感知软件包（输入 n 并回车即可）
 - 无需安装 MATLAB-ROS API（输入 n 并回车即可）
- 安装结束后，我们需要配置运行机械臂程序所需的环境变量。（同样的，以下程序只运行一次即可）

代码块

```
1 echo "source ~/interbotix_ws/install/setup.bash" >> ~/.bashrc
2 source ~/.bashrc
```

- 要检查安装是否成功，我们可以查看 Interbotix 的 ROS 软件包列表。

代码块

```
1 ros2 pkg list | grep interbotix
```

- 它将显示类似下面的列表。

代码块

```
1 interbotix_common_modules
2 interbotix_common_sim
3 interbotix_common_toolbox
4 interbotix_ros_xsarms
5 interbotix_ros_xsarms_examples
6 interbotix_ros_xseries
7 interbotix_tf_tools
8 interbotix_xs_driver
9 interbotix_xs_modules
10 interbotix_xs_msgs
11 interbotix_xs_ros_control
12 interbotix_xs_rviz
13 interbotix_xs_sdk
14 interbotix_xs_toolbox
15 interbotix_xsarm_control
```



```
16 interbotix_xsarm_descriptions
17 interbotix_xsarm_dual
18 interbotix_xsarm_joy
19 interbotix_xsarm_moveit
20 interbotix_xsarm_moveit_interface
21 interbotix_xsarm_ros_control
22 interbotix_xsarm_sim
```

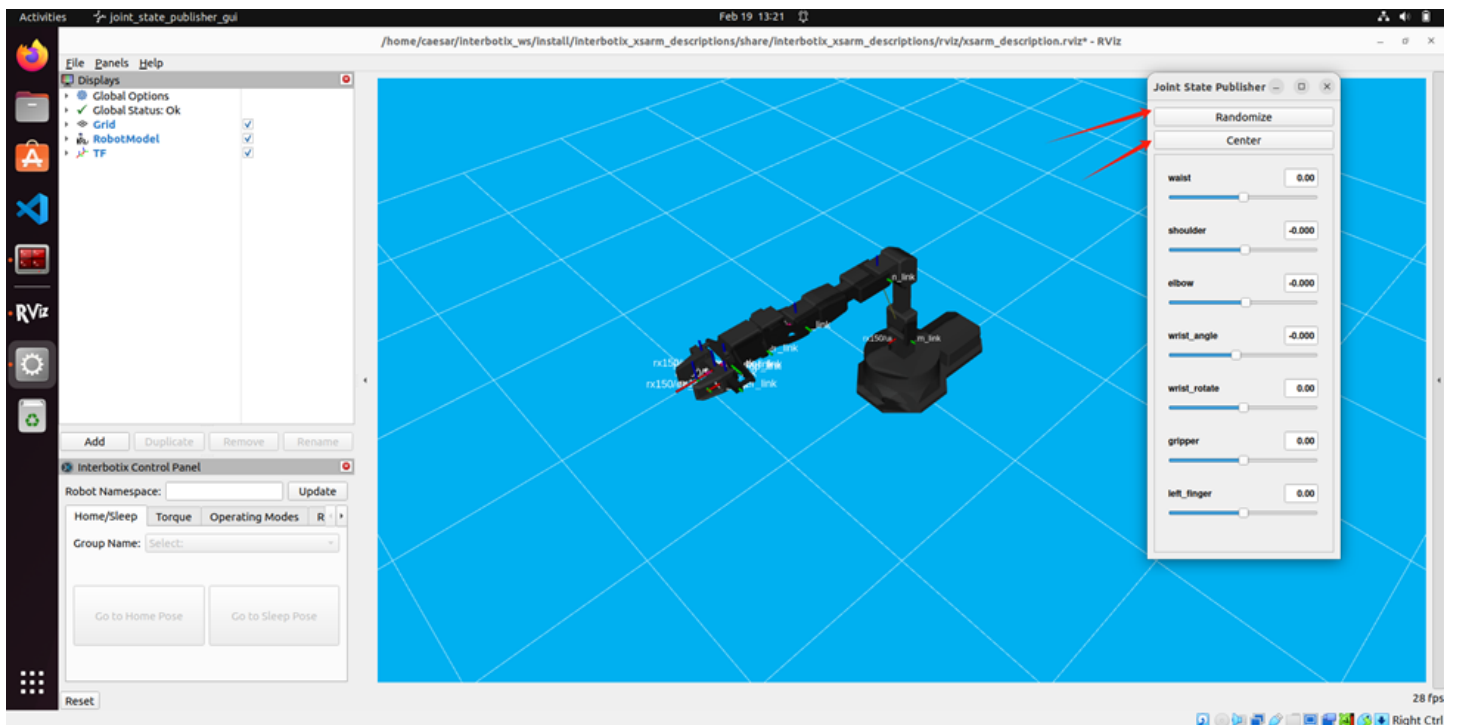
在 Gazebo 中运行机械臂

- 使用以下命令在 Gazebo 中启动 ReactorX 150 机械臂。

代码块

```
1 ros2 launch interbotix_xsarm_descriptions xsarm_description.launch.py
  robot_model:=rx150 use_joint_pub_gui:=true
```

- 启动后，用户可以使用关节状态发布器的图形界面，来手动控制机械臂的关节角度，进行测试。在图形界面上移动滑块就可以实时改变关节的位置。
 - Randomize**：在定义的有效范围内随机设置所有关节的位置。
 - Center**：将所有关节重置为默认或中立位置，通常为零。



💡 这个图形界面的控制器在调试时会很有帮助，尤其是当您想要测试关节角度的正方向时。它的工作原理其实是在用户调整滑块时，把更新后的关节角度发布到 `/joint_states`。

